Part 1

1. **Method**

The objective was to train sequence-to-sequence (Seq2Seq) neural networks to decrypt a substitution-style cipher back into plain English text.

Data Preprocessing: The cipher text was parsed into integer tokens, utilizing a specific rule where '9' represented a space (mapped to token 99) and pairs of digits were parsed as individual character tokens. Tokens were offset to reserve indices for <PAD>, <SOS>, and <EOS>. Max sequence length was truncated to 128 to optimize training speed.

Model Architectures:

Two custom Seq2Seq models were built from scratch using PyTorch:

- **RNN Model:** Utilized standard recurrent cells (ManualRNNCell) for both the encoder and decoder. It incorporated an additive (Bahdanau-style) attention mechanism to help the decoder focus on relevant parts of the input sequence.
- **LSTM Model:** Utilized Long Short-Term Memory cells (ManualLSTMCell) equipped with input, forget, output gates, and cell states. The forget gate biases were initialized to 1.0 to prevent early vanishing gradients. It used a scaled dot-product (Luong-style) attention mechanism.

Training Strategy:

Both models were trained using a 50% teacher-forcing ratio to balance fast convergence and sequence stability. Gradient clipping (max norm = 1.0) was applied to prevent exploding gradients. The LSTM model utilized an Early Stopping callback (patience of 3) to prevent overfitting by monitoring validation loss.

2. Configuration

Both models shared the following core hyperparameter configuration to ensure a fair comparison:

- **Vocabulary:** Max length of 128 tokens
- **Dimensions:** Embedding Dim = 128, Hidden Dim = 256
- **Training setup:** 20 Epochs, Batch Size = 64
- **Optimizer:** Adam Optimizer (Learning Rate = 0.001)
- **Loss Function:** Cross-Entropy Loss (ignoring the <PAD> token index 0)
- **Hardware:** Trained on CUDA

3. Results Obtained

The models were evaluated on an unseen test set based on character-level accuracy, word-level accuracy, and average Levenshtein distance (lower is better for Levenshtein and Loss).

Metric	RNN Model	LSTM Model
Final Test Loss	2.0852	1.1609
Character Accuracy	53.14%	73.70%
Word Accuracy	37.90%	57.15%
Avg Levenshtein Dist.	45.80	26.96

4. Interpretation and Analytical Discussion

Performance Disparity: The LSTM model significantly outperformed the standard RNN model across all evaluation metrics. The LSTM achieved a 38% relative improvement in character accuracy and almost halved the Levenshtein distance compared to the RNN.

RNN's Limitations on Long Sequences:

Looking at the sample outputs for the RNN model, it successfully reconstructs short, simple patterns but falls apart rapidly on longer strings, outputting repetitive gibberish (e.g., repeating "dist in dist in dist"). This is a classic symptom of the **vanishing gradient problem**. Standard RNNs struggle to carry information across long time steps, meaning by the time the decoder reaches the end of a sentence, it has forgotten the contextual cues from the beginning of the cipher.

LSTM's Gating Advantage:

The LSTM resolves this through its gating mechanisms and persistent cell state. By isolating the memory stream from the hidden state stream, the LSTM can decide what information to keep, update, or discard. Initializing the forget gate bias to 1.0 helped the model retain early sequence data effectively right from the first epoch. Consequently, the LSTM samples demonstrate a coherent understanding of English morphology and syntax, even forming mostly correct long words and complete phrases (e.g., predicting "American knowhow" with minor spelling errors).

Conclusion:

For the task of deciphering text where context relies on characters mapped across relatively long sequence lengths, the LSTM is definitively the superior architecture. While the RNN

failed to establish long-term grammatical context, the LSTM paired with dot-product attention successfully reverse-engineered the bulk of the cipher rules.

Here is the report for the second phase of your assignment, focusing on Language Modeling.

Part 2

The second objective shifted from character-level decryption to word-level language modeling to anticipate future transmissions. Two distinct paradigms were implemented and compared:

- **Task 1: Next Word Prediction (NWP) using a State Space Model (SSM).** The dataset was structured causally, where the input is a sequence of words and the target is the sequence shifted by one step into the future. A custom SimplifiedSSMLayer was built using learnable state matrices (A, B, C, D) to model the sequence dynamics recurrently.
- **Task 2: Masked Language Modeling (MLM) using a Bidirectional LSTM (Bi-LSTM).** The dataset randomly replaced 15% of the input tokens with a <MASK> token. A CustomBiLSTM was built from scratch, utilizing dual manual LSTM cells to process the sequence from left-to-right and right-to-left simultaneously, concatenating the hidden states to predict the masked words based on surrounding context.

Training Strategy: A custom vocabulary was built to map the most frequent words to integer indices. Both models were trained utilizing PyTorch's Automatic Mixed Precision (amp.autocast) and a GradScaler to optimize memory usage and computational speed. Outputs were generated using Top-K sampling ($K=5$) while explicitly banning the <UNK> (unknown) token to ensure readable text generation.

2. Configuration

Both language models shared the following core hyperparameters for a balanced comparison:

- **Vocabulary Strategy:** Max 10,000 words, replacing rare words with <UNK>.
- **Dimensions:** Embedding Dimension = 256, Hidden/State Dimension = 256.
- **Regularization:** Dropout = 0.3.
- **Training setup:** 10 Epochs, Batch Size = 64, Adam Optimizer (LR = 0.001).
- **Loss Function:** Cross-Entropy Loss (padding indices and unmasked tokens in MLM were mapped to -100 to be ignored by the loss calculation).

3. Results Obtained

The primary evaluation metric was **Perplexity** (the exponentiation of the Cross-Entropy loss), where a lower score indicates a model is less "surprised" by the unseen test data.

Metric	SSM (Next Word Prediction)	Bi-LSTM (Masked Language Modeling)
Final Perplexity	226.16	127.79

Qualitative Output Examples:

- **NWP (SSM) Prediction:** Input: 'the collective forces will attack the ...' $\rightarrow$ Output: 'Soviet'
- **MLM (Bi-LSTM) Prediction:** Input: 'the collective forces will <MASK> the base' $\rightarrow$ Output: 'take'
- **Continuous Generation (SSM):** 'the collective forces will be a good deal of the greatest of the most of the most of the American continent in the first'

4. Interpretation and Analytical Discussion

Perplexity Disparity: The Custom Bi-LSTM achieved a significantly lower perplexity (127.79) compared to the SSM (226.16). This substantial performance gap is structurally expected due to the nature of the tasks. Masked Language Modeling (MLM) provides the Bi-LSTM with *bidirectional* context—it can look at the words both before and after the <MASK> token to make an informed guess. Conversely, the SSM performing Next Word Prediction is strictly causal; it must predict the future relying solely on the past, making it an inherently more difficult task resulting in higher perplexity.

Generative Capabilities:

The generative tests perfectly illustrate the strengths and weaknesses of the approaches:

1. **Targeted Inference (Bi-LSTM):** The Bi-LSTM excels at "fill-in-the-blank" tasks. Given contextual boundaries ("will [MASK] the base"), it successfully narrows down the grammatical constraints and semantics to predict a highly logical verb ("take").
2. **Open-Ended Generation (SSM):** While the simple diagonal SSM manages to capture local syntactic structures (e.g., forming prepositional phrases like "of the greatest of the most of the..."), it rapidly falls into repetitive loops and loses global semantic coherence over longer generations. This highlights the limitations of a simplified state-space layer without advanced gating or deep attention mechanisms when tasked with sustaining long-term context in open-ended text generation.

Here is the report for the final phase of your assignment, detailing the integration of Language Models for error correction.

Part 3:

1 Method

The final objective was to build a robust pipeline capable of recovering Dominion transmissions degraded by varying levels of noise. The pipeline consists of two stages:

1. **Base Decryption:** The noisy cipher text is passed through the pre-trained Part 1 LSTM Seq2Seq model. Because the input contains noise artifacts, the output is partially corrupted, often containing misspelled or out-of-vocabulary gibberish.
2. **Language Model Correction:** The decrypted text is tokenized and checked against a standard English dictionary. Any "invalid" word triggers the correction algorithm:
    - **Candidate Generation:** The respective Language Model (SSM or Bi-LSTM) predicts the top 500 highly probable words for that position based on the surrounding context.
    - **The Length & Edit Heuristic:** To prevent the model from hallucinating entirely new sentence structures, a strict constraint was enforced: the candidate word *must* have the exact same character length as the corrupted word. The algorithm then selects the candidate that minimizes the Levenshtein (edit) distance to the original noisy word, using the LM's confidence probability as a tie-breaker.

2. Experimental Setup

- **Decryption Model:** LSTM Seq2Seq (trained in Part 1).
- **Language Models:** State Space Model (NWP) and Bi-LSTM (MLM) (trained in Part 2).
- **Data:** Cipher texts subjected to 5 escalating levels of noise (0 through 4).
- **Evaluation Metrics:** Character Accuracy, Word Accuracy, Levenshtein Distance, BLEU (n-gram overlap), and ROUGE-L (longest common subsequence).

3. Results Obtained

The performance of the three approaches—Decryption Alone, Decryption + SSM, and Decryption + Bi-LSTM—was evaluated across the five noise levels.

Key Quantitative Highlights:

- **Zero Noise (Level 0):** Decryption Alone achieved a BLEU of 19.48. Adding the SSM pushed BLEU to 25.91, and Bi-LSTM pushed it to 24.06.
- **Moderate Noise (Level 1):** The baseline decryptor's performance collapsed (BLEU 5.95, Word Acc 29%). The SSM correction rescued the text, achieving a BLEU of 17.56 (a nearly 3x improvement) and Word Acc of 39%.
- **High Noise (Level 4):** Baseline decryption failed completely (BLEU 0.72). The LMs successfully reconstructed a fraction of the semantic meaning, with the SSM achieving a BLEU of 7.99 and Bi-LSTM achieving 6.76.

4. Interpretation and Error Analysis

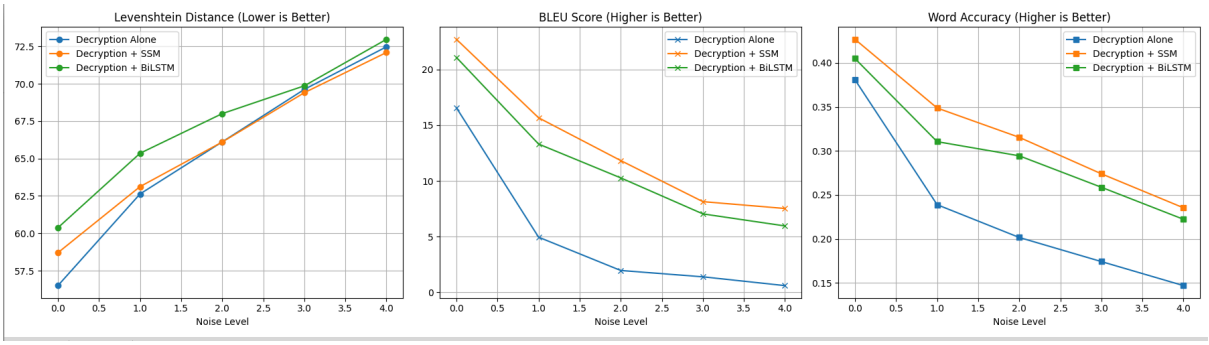
The Semantic Rescue Effect:

Both language models drastically improved Word Accuracy, BLEU, and ROUGE-L scores across all noise levels compared to raw decryption. Interestingly, while the Word Accuracy went up, the Levenshtein distance actually increased slightly (became worse) at lower noise levels when using the LMs. This represents a classic trade-off: the LM sacrifices raw character-by-character matching to force the letters into contextually valid English words. It may change 2-3 characters to fix a word, increasing edit distance, but saving the semantic value of the sentence (which BLEU rewards).

SSM vs. Bi-LSTM Paradox: In Part 2, the Bi-LSTM achieved a significantly lower perplexity than the SSM. However, in this noisy pipeline, **the SSM outperformed the Bi-LSTM across every single noise level** (e.g., +1.8 BLEU difference at Noise Level 1).

- *Why did this happen?* The Bi-LSTM relies heavily on bidirectional context (words to the left *and* right of the target). In a highly noisy sequence, the "future" right-side context is likely also corrupted gibberish, which actively poisons the Bi-LSTM's predictions.
- Conversely, the SSM predicts the next word based sequentially on the *past* context. Because the pipeline evaluates left-to-right, the SSM's "past context" consists of words that have already been cleaned and corrected by the algorithm, giving it a much more stable foundation to make its predictions.

Conclusion: Language modeling is a highly effective post-processing step for noisy decryption. While bidirectional models excel on clean data, causal forward-predicting models (like the SSM) demonstrate greater resilience when iteratively correcting sequences heavily degraded by noise. Furthermore, the strict length-matching heuristic proved highly effective at grounding the language models, preventing them from deleting words or hallucinating completely unaligned text.



Directory Structure of outputs is different. For each task it will create task{number}_{model} directory. In that it will logs fiile for each. Also the models are stored for each respectively.

INLP_A3.

HF HF - part 1